



Functions

dot
LEBANON
Activate Wi-Fi
Go to Settings to activate Wi-Fi

Objectives

Define customized functions

Identify the advantages of functions

Use functions in their coding

Differentiate between functions type

Functions

- Functions are used in any programming language. They are an essential syntax that can simplify and improve your program.
- A function is a set of codes arranged together in a block and given a name. This set of codes will work once you call the block name.
- Every time you write a code, you use two functions. These functions are the “void setup () {}” and the “void loop () {}” each one has a main role in the program. These two functions are doing big tasks in the program while other functions will be responsible for other tasks in the program.

Functions

We can call these functions, custom functions. Once we have them ready we can call them at any time in the “void loop ()”.

The main advantages of functions are:

- Structures programs.
- Smaller programs.
- Easier to understand by other programmers.
- Reduces percentage of error.
- Easier to modify and fix.
- Simplifies complex programs.

Functions

Using a Function:

Using a function can be divided into two steps. The first step is writing the main function, while the second is calling that function.

- Writing a Function:

First, you should specify the type of the function. Then add the initiation parameters. Once in the function, you can write your function code.

Example:

```
void myArray ( /* initial parameters */ ) {
```

```
//Function Code
```

```
}
```

- Calling a Function:

In order to call for a function you need to write the name of this function. Then add the initial parameters if required.

Example:

```
myArray ( /* initial parameters */ );
```

Functions

In the following figure you can find a schematic about function use, and what does each part in a function stands for.

The diagram shows a C function definition with several annotations explaining its parts:

- Datatype of data returned, any C datatype.**: Points to the `int` at the start of the function signature.
- "void" if nothing is returned.**: Points to the `int` at the start of the function signature.
- Function name**: Points to `myMultiplyFunction`.
- Parameters passed to function, any C datatype.**: Points to the parameters `(int x, int y)`.
- Return statement, datatype matches declaration.**: Points to the `return result;` statement inside the function body.
- Curly braces required.**: Points to the opening and closing curly braces `{` and `}`.

```
int myMultiplyFunction(int x, int y){  
    int result;  
    result = x * y;  
    return result;  
}
```

Functions

The table below shows some types of functions:

Function	Definition	Example
<pre>void functionName (input) { Comments }</pre>	A void function: • Will allow the execution of a set of commands, once it is called. • Can take input variables In the Example: the program will print the sensorValue connected to digitalpin 7, by using a function.	<pre>void sensorValue () { int value = digitalRead(7); Serial.print(" Sensor value is"); Serial.println(value); } void loop () { sensorValue (); }</pre>

Functions

<pre>int functionName (input) {</pre> Comments <pre>}</pre>	An int function: • Will allow the execution of a set of commands, once it is called. • Can take input variables • Can return an integer number In the Example: the program will print the sensorValue connected to digitalpin 7, By using a function then return the value of the sensor.	<pre>int sensorValue () { int value = digitalRead(7); Serial.print(" Sensor value is"); Serial.println(value); return (value); } void loop () { Serial.print(" The value is "); Serial.println(sensorValue ()); }</pre>
--	---	--

Functions

<pre>Bool functionName(input){</pre> Comments	A Boolean function: • Will allow the execution of a set of commands, once it is called. • Can take input variables • Can return a Boolean variable	<pre>void loop() { Serial.print(" The value is "); Serial.println(sensorValue ()); }</pre>
<pre>}</pre>	In the Example: the program will print the sensorValue connected to digitalpin 7, by using a function then it will return a Boolean variable.	<pre>} bool sensorValue () { int value = digitalRead(7); bool Test = (value > 20); return (Test); }</pre>

Calling a function

How to think before Connecting

- b. if the Button is pressed, call the FlashingLED function.
- c. Else, call the steadyLED function.

3- In the "flashingLED" function we will :

- a. Use a for loop, that will repeat 50 times in 1 sec such that the LED will change state each 20 ms.
- b. Set the LED signal to HIGH, then delay 20 ms, then set the LED signal to LOW, finally delay 20 ms.

4- In the "steadyLED" function we will send a HIGH signal to the LED.



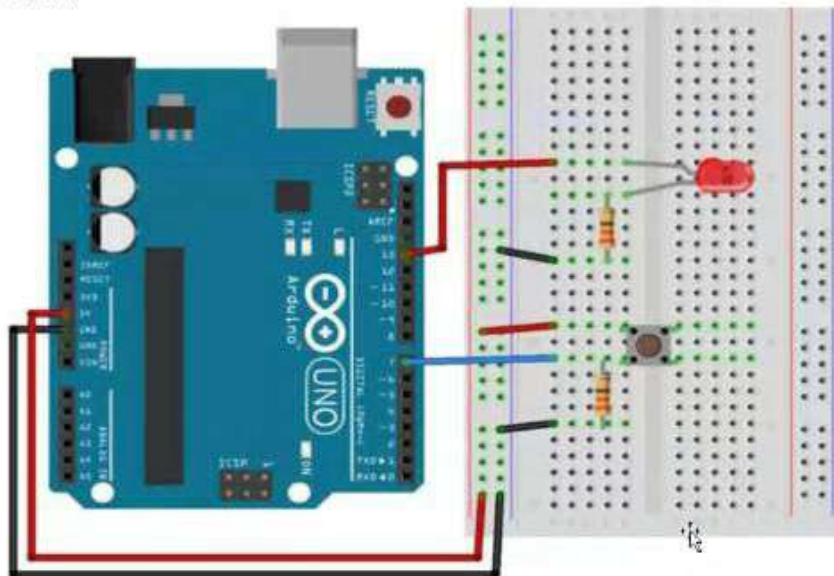
Calling a function

Required Elements:



Board connection

Circuit Connection:



```
Arduino UNO
sketch_jul24a.ino

11 // put your main code here, to run repeatedly:
12 int ButtonState=digitalRead(button);
13 if(ButtonState==HIGH){
14     FlashingLed();
15 }else{
16     SteadyLed();
17 }
18 }
19
20 void SteadyLed(){
21     digitalWrite(led,HIGH);
22 }
23
24 void FlashingLed(){
25     for(int i=0;i<5;i++){
26         digitalWrite(led,HIGH);
27         delay(100);
28         digitalWrite(led,LOW);
29         delay(100);
30     }
```

Click to add text

How to think before Connecting

In this activity, we will present the state of a push button whether it is pressed or not, on the monitor screen, every 3 seconds by using a function.

Programming steps:

1- In the void setup, we want to display the state of the button on the serial monitor, this requires:

- a. Add the Serial Monitor starting order, "Serial.begin(9600);"
- b. Define the needed pins, LED pin as an OUTPUT and the button pin as an INPUT

2- In the void loop, we want to test the button state every 3 seconds and display it. so we will: a. call a function called "buttonState();"

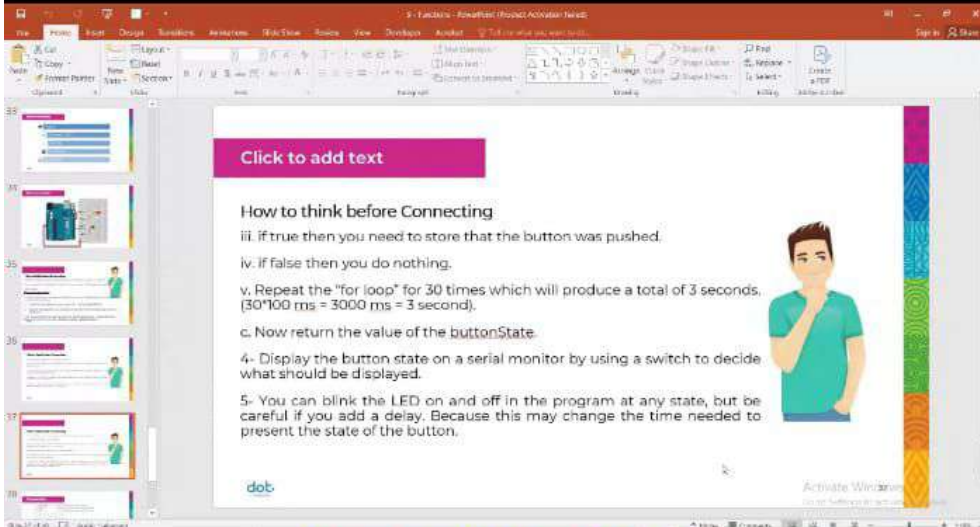


4:20 PM

39.5 K/S 4G 50



Mallak +23



Mohamad is presenting



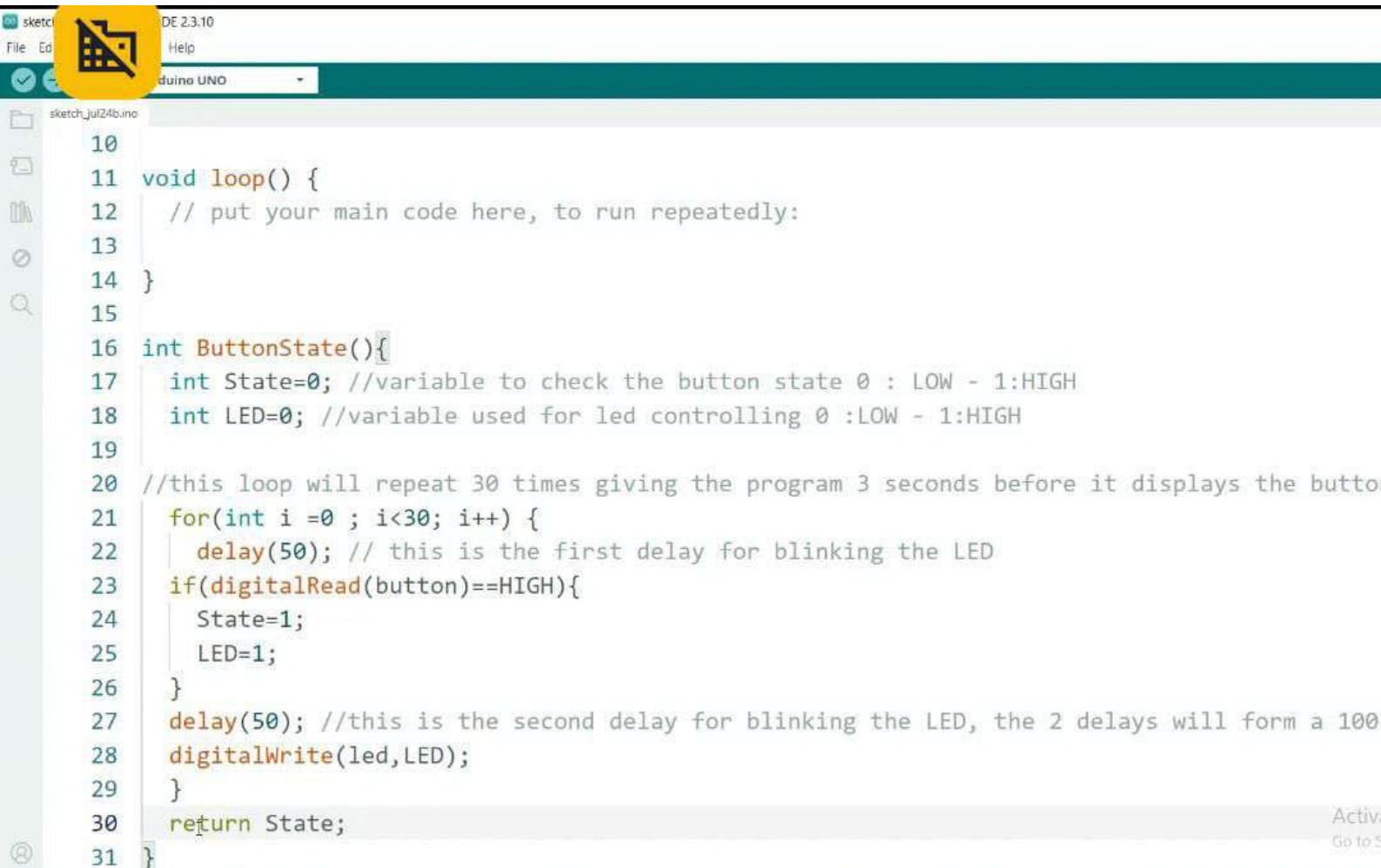
You



Mohamad

21 others





The image shows the Arduino IDE interface. The top menu bar includes 'File', 'Edit', and 'Help'. The 'File' menu is highlighted with a yellow circle. Below the menu bar, the board is set to 'Arduino Uno'. The main workspace displays the following C++ code:

```
10
11 void loop() {
12     // put your main code here, to run repeatedly:
13
14 }
15
16 int ButtonState(){
17     int State=0; //variable to check the button state 0 : LOW - 1:HIGH
18     int LED=0; //variable used for led controlling 0 :LOW - 1:HIGH
19
20     //this loop will repeat 30 times giving the program 3 seconds before it displays the button
21     for(int i =0 ; i<30; i++) {
22         delay(50); // this is the first delay for blinking the LED
23         if(digitalRead(button)==HIGH){
24             State=1;
25             LED=1;
26         }
27         delay(50); //this is the second delay for blinking the LED, the 2 delays will form a 100ms
28         digitalWrite(led,LED);
29     }
30     return State;
31 }
```

On the right side of the IDE, there is a status bar with the text 'Active' and 'Go to 5'.

Board connection

